

Physically Informed Neural Networks pour la modélisation de la diffusion thermique instationnaire 2D dans une pièce

Livrables attendus : Dépôt Git (code propre et documenté), Présentation de projet (5 minutes) et Démonstrateur interactif.

1 Contexte et Objectifs

Les **Physics-Informed Neural Networks (PINNs)** permettent d'intégrer des équations aux dérivées partielles (EDP) directement dans la fonction de perte d'un réseau de neurones. Contrairement à un modèle *data-driven* classique, un PINN peut apprendre un champ physique en respectant les lois de la conservation, même avec très peu (ou pas) de données mesurées à l'intérieur du domaine.

Scénario physique

L'objectif est de prédire la diffusion de la chaleur au cours du temps t dans un domaine bidimensionnel $\Omega = [0, L_x] \times [0, L_y]$ (1 m $\times$ 1 m) :

- Initialement ($t = 0$), la pièce et ses murs sont à une température homogène $T_{\text{amb}} = 20^\circ\text{C}$.
- À $t = 0$, on place au centre de la pièce un objet chaud (pavé ou disque) à $T_{\text{obj}} = 80^\circ\text{C}$.
- Les murs extérieurs restent maintenus à $T_{\text{amb}} = 20^\circ\text{C}$ pour tout $t > 0$ (conditions aux limites de Dirichlet).
- Le réseau doit prédire $T(x, y, t)$ en tout point et à tout instant $t \in [0, t_{\text{max}}]$.

2 Modélisation Mathématique

L'équation qui régit le phénomène est l'**équation de la chaleur 2D instationnaire** sans source interne ($t > 0$) :

$$\frac{\partial T}{\partial t} - \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) = 0 \quad \text{dans } \Omega \times]0, t_{\text{max}}] \quad (1)$$

où α représente la diffusivité thermique du milieu (ex : air, $\alpha \approx 2 \times 10^{-5} \text{ m}^2/\text{s}$, à adimensionner pour stabiliser l'entraînement).

Conditions initiales (IC) et aux limites (BC)

- **Condition initiale** ($t = 0$) :

$$T(x, y, 0) = \begin{cases} T_{\text{obj}} & \text{si } (x, y) \in \text{Objet} \\ T_{\text{amb}} & \text{sinon} \end{cases} \quad (2)$$

- **Conditions aux limites** ($\forall t \in [0, t_{\text{max}}]$) :

$$T(x, y, t) = T_{\text{amb}} \quad \text{sur } \partial\Omega \quad (3)$$

3 Travail à réaliser

1 : Adimensionnement et Échantillonnage

Reformuler l'équation différentielle sous forme adimensionnée. Générer les points de collocation : N_{IC} (condition initiale), N_{BC} (limites) et N_{res} (résidu interne de l'EDP).

2 : Architecture du PINN et Dérivation Automatique

Implémenter le réseau $T_\theta(x, y, t)$ sous PyTorch. Calculer les dérivées partielles via `torch.autograd` et définir la fonction de perte multi-objectif :

$$\mathcal{L}(\theta) = w_{\text{IC}}\mathcal{L}_{\text{IC}} + w_{\text{BC}}\mathcal{L}_{\text{BC}} + w_{\text{res}}\mathcal{L}_{\text{res}} \quad (4)$$

3 : Stratégie d'Entraînement et Optimisation

Mettre en place un entraînement hybride (Adam puis L-BFGS). Tester au moins une technique avancée pour la raideur de la condition initiale (ex : *Residual Adaptive Resampling*, pondération dynamique des pertes ou *Hard-constraints*).

4 : Validation Numérique

Développer un solveur classique (Différences Finies ou Éléments Finis) pour obtenir une solution de référence et évaluer la précision du PINN (MSE, L2-relative error).

5 : Démonstrateur Interactif

Créer une interface (Gradio) permettant de faire varier les paramètres de l'objet, de visualiser la carte de chaleur prédite en temps réel via un slider temporel t , et d'afficher les cartes d'erreur.

4 Ressources

- Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). *Physics-informed neural networks : A deep learning framework for solving forward and inverse problems*. Journal of Computational Physics.
- Documentation PyTorch Autograd (`torch.autograd.grad`).